

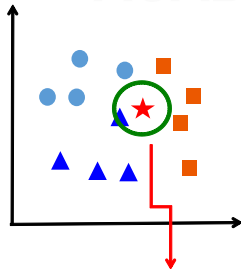
Chapter 02 .scikit-learn KNN 분류 및 회귀 (k-Nearest Neighbor)

- KNN(k-최근접 이웃) 알고리즘 개념
- K-최근접 이웃 분류_생선 종류 분류
- k-최근접 이웃 회귀_생선 무게 예측

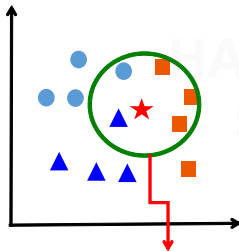
sklearn(사이킷런 : scikit-learn)

- KNeighborsClassifier() : k-최근접 이웃 분류 알고리즘

현재 데이터를 특정값으로 분류하기 위해 기존의 데이터와 현재 데이터를 비교함으로써
현재 데이터와 가장 인접한 k개의 데이터를 선정하여 k개의 레이블 중 가장 많이 분류된
값으로 현재 데이터를 분류



K = 1 일 경우 ★ 는 가까운 ▲ 로 분류



K = 5 일 경우 ★ 는 가까운 데이터 중
가장많은 레이블인 ■ 로 분류

sklearn(사이킷런 : scikit-learn)

- k-최근접 이웃 **분류 알고리즘** 장/단점

장점	단점
- 숫자로 구분된 속성에 거리 기반 knn은 높은 정확도와 성능 발휘 - 오류 데이터가 결과에 영향을 미치지 않음 - 구현 및 알고리즘이 비교적 쉬움 - 샘플 수가 많을 때 좋은 분류 발휘 - 예측하는 시점에 모든 기존 데이터와 거리를 계산하기 때문 모델을 따로 학습시킬 필요 없음(게이른 학습)	- 예측 속도가 느림 - fit() 동작 시 학습데이터를 모두 메모리에 저장하는 특징으로 메모리를 많이 사용하게 되어 고사양의 하드웨어 요구 - 최적의 k를 선택하기 어려움



sklearn(사이킷런 : scikit-learn)

- KNeighborsClassifier() : k-최근접 이웃 분류 모델 클래스

n-neighbors 매개변수 : 이웃의 개수 지정, 기본값 5

fit() : 사이킷런 모델 훈련 메서드, 처음 두 매개변수로 훈련에 사용할 특성 데이터와 타깃(목표) 데이터를 전달, 훈련 입력 데이터 2차원 배열 기대
knn 경우 예측시 이웃을 찾을 수 있도록 단순히 데이터를 메모리에 저장하는 역할이 전부이며 실제 예측 시(predict) 거리 계산 후 판단 ★

score() : 훈련된 모델의 성능 측정, 처음 두 매개변수로 테스트에 사용할 특성 데이터와 타깃(목표) 데이터를 전달
먼저 내부 predict() 예측을 수행한 후 분류 모델의 경우 타깃(목표)와 비교하여 올바른 예측 개수의 비율을 반환, 평가 입력 데이터 2차원 배열 기대 (predict() 와 accuracy_score() 연결하여 정확도 계산)

predict() : 모델을 훈련하고 특정 데이터의 결과를 예측할 때 사용,
특성 데이터 하나만 매개변수로 전달, 2차원 배열 기대

KNeighborsClassifier 분류 모델
기본값 5개의 이웃 데이터를 기준으로
주위의 다수를 차지하는 데이터를 정답으로 예측

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
# 도미 데이터 ( 캐글 Fish Market 데이터 참조 )
breame_length = [25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7, 31.0, 31.0,
                 31.5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0, 34.0, 34.5, 35.0,
                 35.0, 35.0, 35.0, 36.0, 36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0]
breame_weight = [242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0, 500.0, 475.0, 500.0,
                 500.0, 340.0, 600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0, 685.0, 620.0, 680.0,
                 700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0]
```

```
# 빙어 데이터 ( 캐글 Fish Market 데이터 참조 )
smelt_length = [9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0]
smelt_weight = [6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.4, 12.2, 19.7, 19.9]
```

```
length = breame_length + smelt_length
weight = breame_weight + smelt_weight
```

```
fish_data = [ [l, w] for l, w in zip(length, weight)]  
print(fish_data)  
arr = np.array(fish_data)
```

```
fish_target = [1]*35 + [0]*14  
print(fish_target)
```



도미 : 1 , 빙어 : 0 으로 가정

```
model = KNeighborsClassifier() # 이웃개수 n_neighbors의 기본값 5  
model.fit(fish_data, fish_target) # 훈련
```

```
print( model.score(fish_data, fish_target) )
```

모델 성능 평가

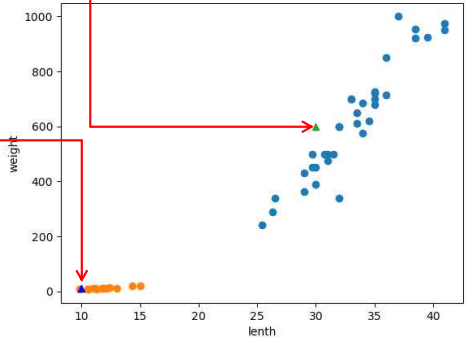
```
print( model.predict([[30, 600],[10,11]]))
```

1.0
[1 0]

빙어로 예측

도미로 예측

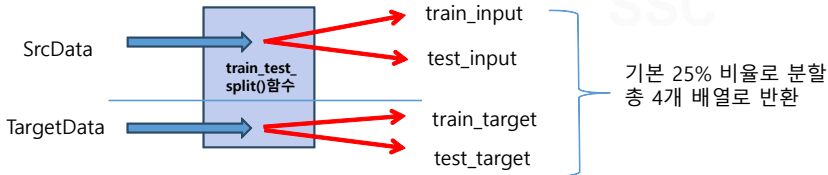
```
plt.scatter(bream_length, bream_weight)
plt.scatter(smelt_length, smelt_weight)
plt.scatter(30, 600, marker='^')
plt.scatter(10, 11, marker='^', c='blue')
plt.xlabel('lenth')
plt.ylabel('weight')
plt.show()
```



sklearn(사이킷런) 훈련 세트 / 테스트 세트 분리

from sklearn.model_selection import train_test_split

- train_test_split() 함수에 전달되는 리스트나 배열을 비율에 맞게 훈련 세트와 테스트 세트 데이터로 분리 및 무작위 데이터 섞어 줌
- random_state 매개변수를 활용해 동일 실험 결과 획득(연습 시 활용)




```
from sklearn.model_selection import train_test_split
```

```
train_input, test_input, train_target, test_target = train_test_split(x, y, test_size,  
random_state, shuffle, stratify)
```

- x : 특성 데이터
- y : 클래스, 레이블
- test_size : 평가 셋의 비율 지정, default = 0.25 (25%)
- random_state : 특정 지정 숫자값에 따라 무작위로 데이터를 섞은 후 분할, 따라서 지정 숫자에 따라 성능 지표값이 상이할 수 있음, 하이퍼 파라미터 튜닝시 이 값을 고정해두고 튜닝해야 매번 데이터셋이 변경되는 것을 방지할 수 있음
- shuffle : 데이터를 분리하기 전 섞을지 지정, default = True
- stratify 매개변수 : 특정 클래스 비율이 불균형 할 경우 타깃 데이터를 지정하여 호출하면 타깃 비율이 유지될 수 있도록 샘플링, 훈련 데이터가 작거나 특정 클래스 샘플이 적을 때 유용

* 반환값

(x_train, x_test, y_train, y_test) : 훈련 데이터, 테스트 데이터, 훈련 레이블, 테스트 레이블 순서의 tuple 반환
(x_train, x_test) : 레이블(y)없이 데이터만 넣을 경우 반환 형식, 데이터만 분리되어 tuple로 반환

#기본25%비율분할

```
train_input, test_input, train_target, test_target = train_test_split(fish_data, fish_target,  
stratify=fish_target, random_state=42)
```

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt

# 도미 데이터
bream_length = [25.4, 26.3, 26.5, 29.0, 29.0, 29.7, 29.7, 30.0, 30.0, 30.7, 31.0, 31.0,
                31.5, 32.0, 32.0, 32.0, 33.0, 33.0, 33.5, 33.5, 34.0, 34.0, 34.5, 35.0,
                35.0, 35.0, 35.0, 36.0, 36.0, 37.0, 38.5, 38.5, 39.5, 41.0, 41.0]
bream_weight = [242.0, 290.0, 340.0, 363.0, 430.0, 450.0, 500.0, 390.0, 450.0, 500.0, 475.0, 500.0,
                500.0, 340.0, 600.0, 600.0, 700.0, 700.0, 610.0, 650.0, 575.0, 685.0, 620.0, 680.0,
                700.0, 725.0, 720.0, 714.0, 850.0, 1000.0, 920.0, 955.0, 925.0, 975.0, 950.0]

# 빙어 데이터
smelt_length = [9.8, 10.5, 10.6, 11.0, 11.2, 11.3, 11.8, 11.8, 12.0, 12.2, 12.4, 13.0, 14.3, 15.0]
smelt_weight = [6.7, 7.5, 7.0, 9.7, 9.8, 8.7, 10.0, 9.9, 9.8, 12.2, 13.4, 12.2, 19.7, 19.9]

fish_length = bream_length + smelt_length
fish_weight = bream_weight + smelt_weight

arr = np.column_stack( ([1,2,3],[5,6,7]) )
print(arr)
```



```
[[1 5]
 [2 6]
 [3 7]]
```

```
fish_data = np.column_stack((fish_length, fish_weight))  
print(fish_data[:5])
```

```
fish_target = np.concatenate((np.ones(35),np.zeros(14)))  
print(fish_target)
```

도미 : 1 , 빙어 : 0 으로 가정

기본 25% 비율 분할

```
train_input, test_input, train_target, test_target = train_test_split(fish_data, fish_target,  
                                                                        stratify=fish_target, random_state=42)
```

```
print(train_input.shape, test_input.shape)  
print(train_target.shape, test_target.shape)  
print(train_target)  
print(test_target)
```

```
knmodel = KNeighborsClassifier()
knmodel.fit(train_input, train_target)
print(knmodel.score(test_input, test_target))
```

[0.]
(빙어로 예측)

```
print(knmodel.predict([[25,150]]))
```

이웃까지의 거리와 이웃 샘플의 인덱스를 반환

[25,150] 데이터 주변 5개 확인

```
distances, indexes = knmodel.kneighbors([[25,150]])
print('distances : ', distances)
print('indexes : ', indexes)
```

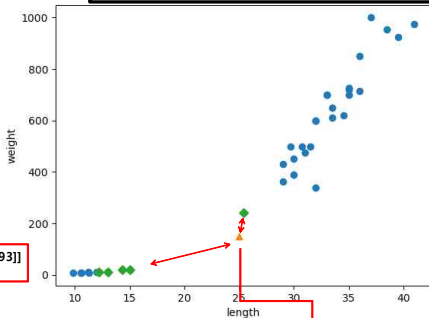
```
distances : [[ 92.00086956 130.48375378 130.73859415 138.32150953 138.39320793]]
indexes : [[21 33 19 30 1]]
```

주변 데이터 산점도

```
plt.scatter(train_input[:,0], train_input[:,1])
plt.scatter(25,150, marker='^')
plt.scatter(train_input[indexs,0], train_input[indexs,1], marker='D')
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```

특성 데이터를
동일 스케일로 맞춰주는
표준화 작업 필요

- 길이와 무게 데이터를 입력 특성데이터로 사용
- 1(도미), 0(빙어) 타깃데이터로 활용해 훈련
- 예측하고자 하는 데이터의 주변 5개를 확인



length 와 weight 비율 차이로
[25,150] 데이터를 빙어에 가까운
데이터로 잘못된 인식 문제 발생

특성 스케일(scale) 전처리

- 표준 점수 활용

- 표준 점수 : 각 특성값이 0에서 표준편차의 몇 배 만큼 차이 나는지 나타내는 지표
- 계산 방법 : (각 특성값 - 평균) / 표준편차 ==> (모든 값을 평균이 0, 분산이 1인 정규 분포로 변환)
- 평균 : `np.mean()` , 표준편차 : `np.std()`

axis = 0

train_input
Data Set
(랜덤 섞여
있는 상태)

axis = 1

```
[[ 29.7  500.]  
 [ 12.2  12.2]  
 [ 33.   700.]  
 [ 11.3   8.7]  
 [ 39.5  925.]  
 [ 29.   430.]  
 [ 36.   714.]  
 [ 36.   850.]  
 [ 31.   475.]  
 [ 35.   720.]  
 ...]
```

```
mean = np.mean(train_input, axis = 0 )  
std = np.std(train_input, axis = 0 )  
print(mean, std)
```

mean : [27.29722222 454.09722222]

std : [9.98244253 323.29893931]

특성 스케일(scale) 전처리

- MinMaxScaler 구현

- (모든 값을 0 ~ 1 사이의 실수로 변환)
- 계산 방법 : (각 특성값 - 최소값) / (최대값 - 최소값)
- 최대값 : np.max() , 최소값 : np.min()

train_input
Data Set
(랜덤 섞여
있는 상태)

axis = 0

axis = 1

[[29.7 500.]
[12.2 12.2]
[33. 700.]
[11.3 8.7]
[39.5 925.]
[29. 430.]
[36. 714.]
[36. 850.]
[31. 475.]
[35. 720.]
...

```
train_max = np.max(train_input,axis=0)
train_min = np.min(train_input,axis=0)
# (특성데이터 - 최소값) / ( 최대값-최소값 )
max_min_sub=train_max - train_min
```

```
train_scaled=(train_input - train_min) / max_min_sub
```

```
print(train_scaled)
```

[[0.638 0.497]
[0.077 0.006]
[0.744 0.698]
[0.048 0.002]
.....

train_input
Data Set

```
[[ 29.7 500.]  
 [ 12.2 12.2]  
 [ 33. 700.]  
 [ 11.3 8.7]  
 [ 39.5 925.]  
 [ 29. 430.]  
 [ 36. 714.]  
 [ 36. 850.]  
 [ 31. 475.]  
 [ 35. 720.]  
 ..
```

평균 빼기

```
train_scaled=train_input - mean  
print(train_scaled)
```

평균 : [27.29722222 454.09722222]

=

결과

```
[[ 2.40277778 45.90277778]  
 [-15.09722222 -441.89722222]  
 [ 5.70277778 245.90277778]  
 [-15.99722222 -445.39722222]  
 [ 12.20277778 470.90277778]  
 [ 1.70277778 -24.09722222]  
 [ 8.70277778 259.90277778]  
 [ 8.70277778 395.90277778]
```

평균 뺀 결과

```
[[ 2.40277778 45.90277778]
 [-15.09722222 -441.89722222]
 [ 5.70277778 245.90277778]
 [-15.99722222 -445.39722222]
 [ 12.20277778 470.90277778]
 [ 1.70277778 -24.09722222]
 [ 8.70277778 259.90277778]
 [ 8.70277778 395.90277778]
 ...]
```



나누기

```
std : [ 9.98244253 323.29893931 ]
```



결과

표준편차 나누기

```
train_scaled=train_scaled / std
print(train_scaled)
```

```
[[ 0.24070039 0.14198246]
 [-1.51237757 -1.36683783]
 [ 0.5712808 0.76060496]
 [-1.60253587 -1.37766373]
 [ 1.22242404 1.45655528]
 [ 0.17057727 -0.07453542]
 [ 0.87180845 0.80390854]
 [ 0.87180845 1.22457184]
 [ 0.37092904 0.06465464]
 ...]
```



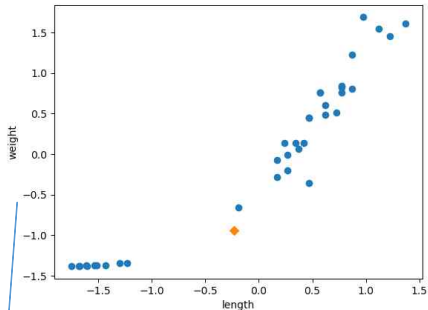
```
# train_input 데이터 셋 표준점수 로 scale 조정  
# 표준 점수 정규화: (데이터 셋 - 평균) / 표준편차  
mean = np.mean(train_input, axis=0)  
std = np.std(train_input, axis=0)  
print(mean, std)
```

```
train_scaled = train_input - mean  
print(train_scaled)
```

```
train_scaled = train_scaled / std  
print(train_scaled)
```

```
# 임의의 데이터 셋도 scale 조정  
new = ([25, 150] - mean) / std  
print('new : ', new)
```

```
# scale 조정 데이터 산점도  
plt.scatter(train_scaled[:,0], train_scaled[:,1])  
plt.scatter(new[0], new[1], marker='D')  
plt.xlabel('length')  
plt.ylabel('weight')  
plt.show()
```



scale 조정 됨

- Scale 조정된 train_scaled 데이터 셋으로 **훈련**
- 모델 평가(score)시 동일 스케일 조정한 테스트 데이터 셋 (test_scaled) 활용
모델 평가
- **new** 데이터 예측

#모델훈련

```
knmodel=KNeighborsClassifier()  
knmodel.fit( train_scaled, train_target)
```

#테스트 데이터셋스케일 조정

```
test_scaled=(test_input-mean)/std
```

#모델평가

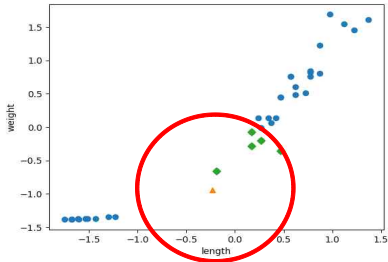
```
print(knmodel.score( test_scaled, test_target))
```

#예측

```
print('예측: ',knmodel.predict([new])) #[1.]도미로예측
```

1.0
예측 : [1.]

```
#new데이터의k-최근접이웃계산:kneighbors()함수활용
#k-최근접이웃까지산점도출력
distances,indexes=knmodel.kneighbors([new])
print('k-최근접이웃인덱스:',indexes)
#scale조정데이터산점도
plt.scatter(train_scaled[:,0],train_scaled[:,1])
plt.scatter(new[0],new[1],marker='^')
plt.scatter(train_scaled[indexes,0],train_scaled[indexes,1],marker='D')
plt.xlabel('length')
plt.ylabel('weight')
plt.show()
```

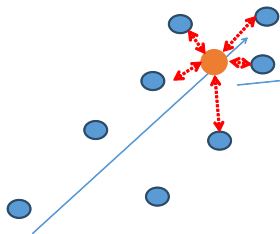


sklearn(사이킷런 : scikit-learn)

- **KNeighborsRegressor()** : k-최근접 이웃 **회귀** 모델 클래스

- 임의의 어떤 숫자를 예측하는 문제
- 지속적으로 변하는 부동산 가격, 경제 성장률 등을 예측
- k-최근접 이웃 데이터의 **평균 계산 한 예측 값을 이용해**
모델 성능을 결정계수(R^2)로 평가

분류 경우 혼동 행렬 사용해 모델 성능 평가하겠지만 **회귀인 경우**
다른 평가 지표 사용 ★



최근접 이웃 데이터
평균 계산

예제)
놓어 길이에 따른 이웃
데이터의 무게 평균 계
산하여 예측

회귀 모델 성능 평가 지표

: 회귀의 평가를 위한 지표는 실제 값과 회귀 예측값의 차이 값을 기반으로 한 지표가 중심

- MAE (Mean Absolute Error) : 평균 절대값 오차
- MSE (Mean Squared Error) : 평균 제곱 오차
- RMSE (Root Mean Squared Error) : 평균 제곱 오차의 루트
- 결정계수 (R^2)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

k-최근접 이웃 회귀 모델 : score() 함수 : 결정계수 값 반환

$$R^2 = 1 - \frac{\sum (\text{타겟} - \text{예측})^2}{\sum (\text{타겟} - \text{평균})^2}$$

→ (타겟 과 예측한 값의 차이를 제곱)

→ (타겟 과 타겟 평균의 차이를 제곱)

모델의 성능을 직감적으로 파악하기 어려울 경우 다음 예제 처럼 **평균 절대값 오차** 사용해 확인 가능

- 타겟의 평균 정도를 예측하는 수준이라면 (분자 와 분모가 비슷해짐) R^2 는 0에 가까워 짐
- **예측이 타겟과 아주 가까워 지면** (분자가 0에 가까워짐) R^2 는 1에 가까워 짐
- R^2 이 **1에 가까울 수록 좋은 모델 성능 평가**

회귀모델 활용 -> 농어 길이에 따른 무게 예측을 예측!!

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsRegressor
```

```
# 농어 길이 데이터 ( 캐글 Fish Market 데이터 참조 )
```

```
perch_length=np.array([8.4,13.7,15.0,16.2,17.4,18.0,18.7,19.0,19.6,20.0,21.0,  
21.0,21.0,21.3,22.0,22.0,22.0,22.0,22.0,22.5,22.5,22.7,  
23.0,23.5,24.0,24.0,24.6,25.0,25.6,26.5,27.3,27.5,27.5,  
27.5,28.0,28.7,30.0,32.8,34.5,35.0,36.5,36.0,37.0,37.0,  
39.0,39.0,39.0,40.0,40.0,40.0,40.0,42.0,43.0,43.0,43.5,  
44.0])
```

```
# 농어 무게 데이터 ( 캐글 Fish Market 데이터 참조 )
```

```
perch_weight=np.array([5.9,32.0,40.0,51.5,70.0,100.0,78.0,80.0,85.0,85.0,110.0,  
115.0,125.0,130.0,120.0,120.0,130.0,135.0,110.0,130.0,  
150.0,145.0,150.0,170.0,225.0,145.0,188.0,180.0,197.0,  
218.0,300.0,260.0,265.0,250.0,250.0,300.0,320.0,514.0,  
556.0,840.0,685.0,700.0,700.0,690.0,900.0,650.0,820.0,  
850.0,900.0,1015.0,820.0,1100.0,1000.0,1100.0,1000.0,  
1000.0])
```

데이터셋 분리

회귀모델 활용 -> 농어 길이에 따른 무게 예측



특성 : perch_length (농어 길이)

타겟 : perch_weight (농어 무게)

```
train_input, test_input, train_target, test_target = \
    train_test_split(perch_length, perch_weight, random_state=42)
```

```
print(train_input.shape)
```

```
print(test_input.shape)
```

사이킷런 훈련을 위해 2차원 배열로 변환

```
train_input = train_input.reshape(-1,1) # 넘파이 배열 크기 자동 지정 : -1
```

```
test_input = test_input.reshape(-1,1)
```

```
print(train_input.shape)
```

```
print(test_input.shape)
```

(42,)
(14,)

(42, 1)
(14, 1)

KNR_model = KNeighborsRegressor() # K-최근접 이웃 회귀 모델

#훈련

KNR_model.fit(train_input,train_target)

#평가

print(KNR_model.score(test_input, test_target)) # 결정계수(R^2)로 평가

#오차율 계산 : 예측과 타깃의 절대값 오차 평균 계산

from sklearn.metrics import mean_absolute_error

테스트셋 예측

test_predicted = KNR_model.predict(test_input)

print(test_predicted)

테스트셋 예측 데이터와 테스트셋 타깃 평균 절대값 오차 계산

mae = mean_absolute_error(test_target, test_predicted)

print(mae) # 약 19g 오차

훈련데이터 평가

print(KNR_model.score(train_input, train_target))

(테스트 데이터 모델 성능 평가 점수)

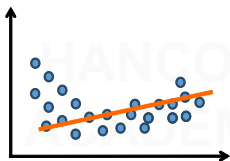
0.992809406101064

(훈련 데이터 모델 성능 평가 점수)

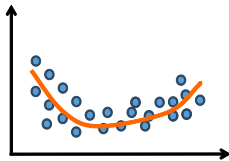
0.9698823289099254

훈련 데이터 성능이 테스트 데이터 성능 보다 떨어지는 과소 적합 발생

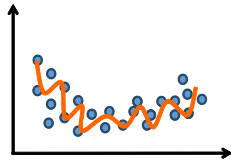
과대 적합(overfitting)	과소 적합(underfitting)
- 훈련 세트 성능 > 테스트 세트 성능 - 훈련 세트 성능이 월등히 높은 경우 테스트 세트 나 임의의 새로운 샘플 에 대한 예측 미비	- 훈련 세트 성능 < 테스트 세트 성능 - 훈련 세트, 테스트 세트 성능 모두 저하  - 모델이 단순하여 적절한 훈련을 못한 경우
해결방법 - dropout, 적은 뉴런수 선택으로 파라미터 수 가 적은 모델 선택하거나, 모델에 제약을 가해서 단순화 - 훈련 데이터 더 많이 확보 - 훈련 데이터의 잡음을 줄임	해결방법 - 파라미터 수가 더 많은 강력한 모델을 선택 - 학습 알고리즘에 더 좋은 특성을 제공 - 모델의 제약을 줄여 더 복잡한 모델로



과소 적합



올바른 모델



과대 적합

- 훈련 세트 성능 이 테스트 세트 성능 보다 다소 좋으면서 두 성능 차이가 크지 않고 두 데이터 세트 모두 좋은 성능 평가를 갖는 모델 설계가 중요

- k-최근접 이웃 기본값 5개를 3개로 변경하여 과소적합 문제 해결
- `KNR_model.n_neighbors = 3`



모델에 제약을 줄여줌

`KNR_model.n_neighbors = 3`

#훈련

```
KNR_model.fit(train_input, train_target)
```

훈련 데이터 평가

```
print( KNR_model.score(train_input, train_target) )
```

(훈련 데이터 모델 성능 평가 점수)

0.9804899950518966

테스트 데이터 평가

```
print( KNR_model.score(test_input, test_target) )
```

(테스트 데이터 모델 성능 평가 점수)

0.9746459963987609

과소
적합
해결

임의 데이터 예측 : 길이 40 cm 놓어 --> [921.66666667] 크기로 예측

```
print( KNR_model.predict([[40]]))
```

[1033.33333333 1033.33333333]

#새로운 샘플이 훈련데이터의 범위를 벗어나면 예측 오류 한계 존재

```
print(KNR_model.predict([[100],[200]]))
```

```
KNR_model = KNeighborsRegressor(n_neighbors=3) # K-최근접 이웃 회귀 모델
```

```
#훈련
```

```
KNR_model.fit(train_input,train_target)
```

```
# 임의의 데이터 예측
```

```
print( KNR_model.predict([[50]]))
```

[1033.33333333]

```
# 50 데이터에 대한 최근접 3개 이웃 샘플 인덱스 반환  
distances, indexes = KNR_model.kneighbors([[50]])
```

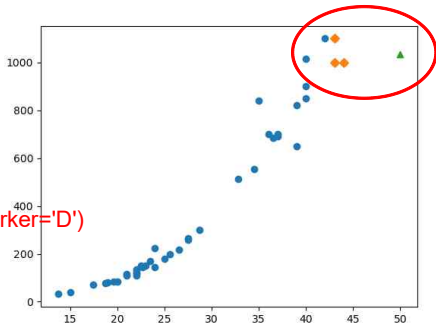
```
plt.scatter(train_input, train_target)
```

```
plt.scatter(train_input[indexes],train_target[indexes], marker='D')
```

```
# 50 데이터 scatter
```

```
plt.scatter(50, 1033, marker='^')
```

```
plt.show()
```



이웃 3개 데이터에 대한 평균 계산으로
50cm 농어에 대한 무게 1033g 예측 오류 한계



문제점 해결 위해
선형회귀, 다항회귀등 다른
회귀 모델 사용 (뒷 내용에서 거론)



** 농구선수 포지션 데이터 활용한 KNN 모델 실습 예제 **

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('basketball_stat.csv')
print(df.head())
print(df.info())
print(df['Pos'].value_counts())
```

```
sns.lmplot(x='TRB',y='3P',data=df, fit_reg=False,
           scatter_kws={'s':150},markers=['o','x'], hue='Pos')
plt.title('TRB and 3P in 2d plane')
plt.show()
```

total
Rebound

('TRB', '3P' 특성 항목을 이
용한 'Pos' 분류 시각화)

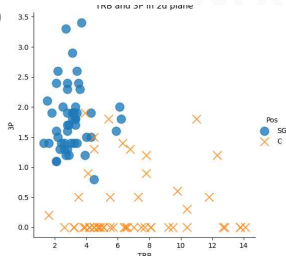
Total Rebound percentage

Block
Percentage

	Player	Pos	3P	2P	TRB	AST	STL	BLK
0	Alex Abrines	SG	1.4	0.6	1.3	0.6	0.5	0.1
1	Steven Adams	C	0.0	4.7	7.7	1.1	1.1	1.0
2	Alexis Ajinca	C	0.0	2.3	4.5	0.3	0.5	0.6
3	Chris Andersen	C	0.0	0.8	2.6	0.4	0.4	0.6
4	Will Barton	SG	1.5	3.5	4.3	3.4	0.8	0.5

```
C    50
SG   50
Name: Pos, dtype: int64
```

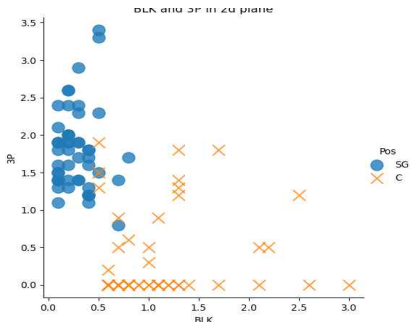
C : 센터
SG : 슈팅가드



1. 농구선수 포지션 데이터 특성 시각화 및 추출

```
sns.lmplot(x='BLK',y='3P',data=df, fit_reg=False,  
           scatter_kws={'s':150},markers=['o','x'], hue='Pos')  
plt.title('BLK and 3P in 2d plane')  
plt.show()
```

('BLK', '3P' 특성 항목을 이용
한 'Pos' 분류 시각화)



#시각화로 '3P', 'TRB', 'BLK' 컬럼 데이터가 Pos 구분에
적절한 데이터로 판단

#데이터 가공 : 불필요한 '2P','AST','STL' 컬럼 데이터 삭제
df.drop(['2P','AST','STL'],axis=1, inplace=True)
print(df.head())

2. 특성 정리된 데이터 활용한 KNN 모델 k-fold 교차 검증 (최적 하이퍼파라미터 찾기 ==> 이웃(k) 개 설정 값)

~~~~~ 앞 PPT 에서 정리된 df 데이터프레임 활용 ~~~~~

# 훈련 / 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
```

```
train, test = train_test_split(df, test_size=0.2, random_state=45) # 20 비율로 훈련셋, 테스트셋 분리
```

```
print(train.shape[0]) # 80
```

```
print(test.shape[0]) # 20
```

# k-fold 교차 검증으로 최적의 Knn 파라미터 ( k 값 ) 찾기

# cross\_val\_score() 활용

# 최적의 k를 찾기위한 교차 검증 수행할 k의 범위를

# 3 부터 학습 데이터 절반까지 설정, 홀수로 설정이 적절

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.model_selection import cross_val_score
```

```
max_k_range = train.shape[0] // 2
```

```
k_list = []
```

```
for i in range(3, max_k_range, 2):
```

```
    k_list.append(i)
```

```
print(k_list) # k 값 리스트
```

[3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27,  
29, 31, 33, 35, 37, 39]

# 데이터 추출

```
x_train = train[['3P','BLK','TRB']]
```

# 훈련데이터 특성

```
y_train = train[['Pos']]
```

# 훈련데이터 타겟

## 2-1) k 값 바뀌가면서 knn 모델 k-fold 교차 검증

```
cross_validation_scores = []
```

```
for k in k_list:
```

```
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    scores = cross_val_score(knn, x_train, y_train.values.ravel(), cv=10, scoring='accuracy')
```

```
    cross_validation_scores.append(scores.mean())
```

```
print(cross_validation_scores)
```

훈련데이터를 10번 교차 검증한:  
정확도 반환

각 k 값에 대한 10번  
의 교차검증에 대한  
평균값을 list에 저장

k-fold 교차검증  
에 대한 별참자료  
참조

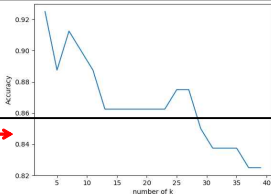
[0.925, 0.8875, 0.9125, 0.9, 0.8875, 0.8625, 0.8625, 0.8625, 0.8625, 0.8625, 0.875, 0.875, 0.85, 0.8375, 0.8375, 0.8375, 0.825, 0.825]

```
plt.plot(k_list, cross_validation_scores)
```

```
plt.xlabel('number of k')
```

```
plt.ylabel('Accuracy')
```

```
plt.show()
```



( k=3 일때 가장 높은  
정확도 )



### 3. test 데이터 활용한 Knn 모델 평가 및 예측

~~~~~ 앞 PPT 에서 정리된 df 데이터프레임 활용 ~~~~~

훈련 / 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
```

```
train, test = train_test_split(df, test_size=0.2, random_state=49) # 20 비율로 훈련셋, 테스트셋 분리
```

```
print(train.shape[0]) # 80
```

```
print(test.shape[0]) # 20
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)
```

데이터 추출

```
x_train = train[['3P','BLK','TRB']]
```

```
y_train = train[['Pos']]
```

KNN 모델 학습

```
knn.fit(x_train, y_train.values.ravel())
```

교차 검증으로 찾은 k 값 3으로 설정

타깃 데이터 1차원 배열로
변환하여 전달

테스트 데이터에서 분류를 위해 사용될 속성을 지정

```
x_test = test[['3P','BLK','TRB']] # x_test 데이터로 예측
```

```
y_test = test[['Pos']] # 분류 실제 값
```

3-1) test 데이터 활용한 Knn 모델 평가 및 예측

테스트 데이터 예측 시작

pred = knn.predict(x_test)

print('예측 결과 : ', pred)

['SG' 'C' 'C' 'C' 'SG' 'C' 'SG' 'SG' 'SG' 'SG' 'SG' 'C' 'SG' 'C' 'SG' 'C'
'C' 'C' 'C' 'C']

모델 예측 정확도 출력

print(y_test.values) # numpy (20,1) 2차원배열

print(y_test.values.ravel()) # numpy 1차원 배열 데이터

from sklearn.metrics import accuracy_score

accuracy_score() : confusion matrix(혼동행렬) 활용 정확도 계산

print('accuracy : ' + str(accuracy_score(y_test.values.ravel(), pred))) # 0.95

print('acc : ', knn.score(x_test, y_test)) # 0.95

실제 데이터와 예측 데이터 Dataframe 변환 출력

comparison = pd.DataFrame({'prediction':pred, 'truth value':y_test.values.ravel()})

print(comparison.head())

| | prediction | truth value |
|---|------------|-------------|
| 0 | SG | SG |
| 1 | C | C |
| 2 | C | C |
| 3 | C | C |
| 4 | SG | SG |

Thank You